

Алгоритмы сортировки данных

Быстрая сортировка Хоара

Обменная сортировка с разделением (быстрая сортировка Хоара)

- ▶ Этот метод предложил Ч. Э. Р. Хоар в 1962 г.
- ▶ Стратегия – «разделяй и властвуй»
- ▶ Суть метода в делении массива (его части) на два фрагмента, а затем сортировки этих фрагментов независимо друг от друга

Быстрая сортировка Хоара

1. В исходном неотсортированном массиве выбрать некоторый элемент $x = a[k]$ (барьерный элемент)
 2. Переставить элементы массива таким образом, чтобы слева от x оказались элементы массива, меньшие или равные x , а справа – элементы массива, большие x
 3. Применить п.1,2 для каждой из этих частей
- И так до тех пор, пока не останутся подмассивы, состоящие из одного элемента, то есть пока не будет отсортирован весь массив

Быстрая сортировка Хоара

Логика:

процедура `quickSort(s, f)`

если $s < f$:

`q = partition(s, f)`

`quickSort(s, q)`

`quickSort(q+1, f)`

Для всего массива вызов:

`quickSort(0, n-1)`

Быстрая сортировка Хоара

Процедура `quick_sort(a, first, last)`:

```
i = first, j = last
x = a[(first + last) // 2]
while i <= j:
    while a[i] < x: i++
    while a[j] > x: j--
    if i <= j:
        a[i] <-> a[j], i++, j--
if (first < j): quick_sort(a, first, j)
if (i < last): quick_sort(a, i, last)
```

Быстрая сортировка Хоара

- ▶ Выбор разделяющего элемента (примеры)

8 12 3 11 19 7 4 16

А) Выбор $x = a[s] = 8$ ($p = 1$)

Б) Выбор $x = a[(s + f) // 2] = a[(1 + 8) // 2] = a[4] = 11$

В) Выбор случайным образом, например $x = a[5] = 19$

А что будет для массива типа 2 2 2 2 2 2 2 2?

Как алгоритм работает с уже отсортированным массивом? Можно ввести проверку того, что не было $\leftrightarrow$, и считать отсутствие $\leftrightarrow$ признаком того, что массив отсортирован?

Эффективность

Наихудший вариант работы алгоритма quickSort. Разбиение порождает одну подзадачу с n элементов, а вторую – с 0 элементов. Для выполнения разбиения требуется время $O(n)$. Рекуррентное соотношение:

$$T(n) = T(n-1) + T(0) + O(n) = T(n-1) + O(n).$$

Арифметическая прогрессия. Результат – $O(n^2)$

Эффективность

Наилучший вариант. В самом благоприятном случае функция Partition делит задачу размером n на две подзадачи, размер каждой из которых не превышает $n/2$.

Рекуррентное соотношение:

$$T(n) \leq 2 \cdot T(n/2) + O(n).$$

Имеем: $T(n) = O(n \cdot \log_2 n)$

Эффективность

В среднем. «Удачный» и «неудачный» выбор разделяющего элемента. Элемент удачный, если при сортировке массива а он попадёт на место с номером от $n/4$ до $3 \cdot n/4$. При разбиении массива по удачному элементу размеры левого и правого массивов будут не больше $3 \cdot n/4$. Вероятность выбора хорошего элемента $1/2$ Рекуррентное соотношение:

$$T(n) \leq O(n) + \frac{1}{n} \sum_{i=1}^n (T(i-1) + T(n-i))$$

Решение $O(n \cdot \log_2 n)$

Анализ алгоритма

- ▶ Общее число сравнений $n \log_2 n$,
общее число обменов $\frac{n \cdot \log n}{6}$
- ▶ Наихудшая производительность метода
будет порядка n^2

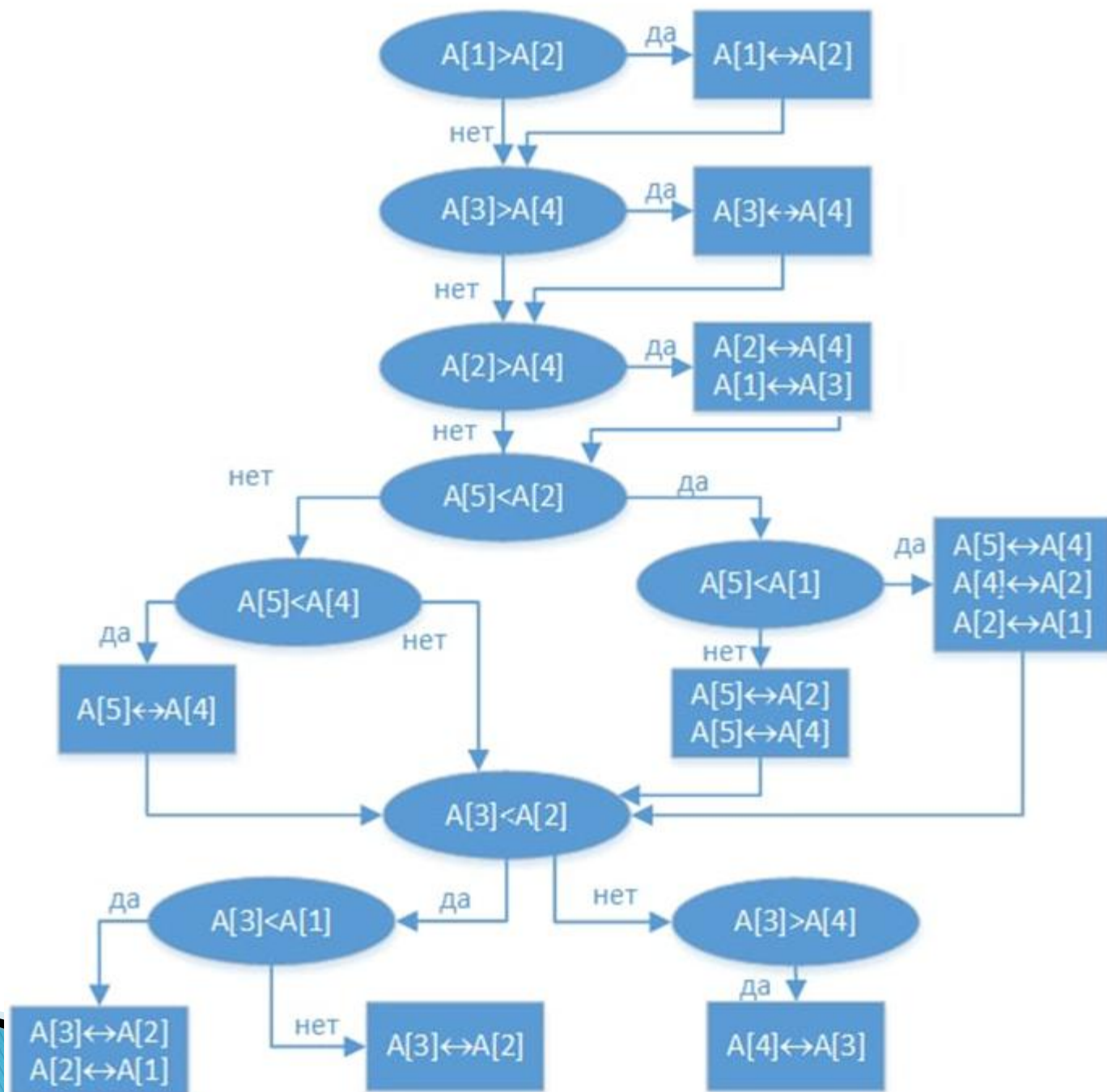
Сортировка 5 элементов за 7 сравнений

Сортировка 5 элементов за 7 сравнений

- ▶ Алгоритм предложен Г. Б. Демутом в 1956 году
- ▶ Используется в более сложных задачах, как элемент, позволяющий конструировать эффективные методы решения
- ▶ Рассмотренные алгоритмы: $5 \cdot \log_2 5$, а это более десяти сравнений

Сортировка 5 элементов за 7 сравнений

- ▶ Количество возможных комбинаций равно числу перестановок из пяти элементов ($5! = 120$). Фактически, отсортировать – значит найти одну комбинацию из 120.
- ▶ Формула Р. Хартли: $I = \log_2 n$
- ▶ Требуется получить 7 бит информации ($I = \log_2 120 \approx 7$). Для представления 120 требуется 7 разрядов.
- ▶ Игра с ответами «да» и «нет». Пять элементов действительно можно отсортировать за 7 сравнений, а за меньшее количество уже нельзя



Программная реализация

```
if A[1]>A[2]: Swap(1,2);  
if A[3]>A[4]: Swap(3,4);  
if A[2]>A[4]: Swap(2,4), Swap(1,3)  
if A[5]<A[2]:  
    if A[5]<A[1]:  
        Swap(5,4), Swap(4,2), Swap(2,1)  
    else: Swap(2,5), Swap(4,5)  
else if: A[5]<A[4]: Swap(5,4)  
if A[3]<A[2]:  
    if A[3]<A[1]:  
        Swap(2,3), Swap(1,2)  
    else: Swap(2,3)  
else if A[3]>A[4]: Swap(3,4)
```


Сортировка подсчетом

Сортировка подсчетом

- ▶ Пример: $A[10, 5, 11, -5, 1, -4, 13, 12, -4, 13]$
- ▶ Сформируем дополнительный массив счетчиков (*count*), в котором каждый i -тый элемент равен количеству элементов A , меньших $A[i]$
- ▶ *count* $[5, 4, 6, 0, 3, 1, 8, 7, 2, 9]$
- ▶ При использовании дополнительного массива b для хранения результата, остается переписать каждый элемент исходного массива на соответствующее место в результирующем массиве

Сортировка подсчетом

- ▶ процедура **sortAccount**:

для i **от** 1 **до** n : $\text{count}[i]=0$

для i **от** n **до** 2:

для j **от** $i-1$ **до** 1:

если $a[i]<a[j]$: $\text{count}[j]++$

иначе: $\text{count}[i]++$

для i **от** 1 **до** n : $b[\text{count}[i]+1]=a[i]$

- ▶ Временная сложность $O(n^2)$

Сортировка подсчетом

```
▶ процедура sortAccount: {без доп.массива b}  
  для i от 1 до n: count[i]=1  
  для i от n до 2:  
    для j от i-1 до 1:  
      если a[i]<a[j]: count[j]++  
      иначе: count[i]++  
  для i от 1 до n:  
    пока count[i]<>i:  
      a[i]↔a[count[i]]  
      count[i]↔count[count[i]]
```

Сортировка подсчетом

- ▶ Смысл алгоритма со сложностью $O(n^2)$ в том, что его идея применима при ограничениях на диапазон значений элементов
- ▶ Пример: 2, 4, 3, 2, 4, 2, 3, 4, 3, 2
- ▶ Подсчитаем, сколько раз каждое значение встречается в массиве
- ▶ массив count: 4, 3, 3

Сортировка подсчетом

► процедура **sortAccount**: $\{[u, v] - \text{диапазон возможных значений}\}$

для i **от** u **до** v : $\text{count}[i] = 0$

для i **от** 1 **до** n :

$\text{count}[a[i]]++$

$q = 1$

для i **от** u **до** v :

для j **от** 1 **до** $\text{count}[i]$:

$a[q] = i, q++$

► Временная оценка алгоритма – $O(n)$

Пирамидальная сортировка

- ▶ Пирамида, строится за время $O(n \cdot \log_2 n)$, при этом максимальный элемент является первым в массиве
- ▶ Поменяем его местами с последним элементом массива, и он окажется на своем месте, на его место запишем последний элемент, а затем «протолкнем» его по пирамиде, но она уже состоит не из n элементов, а из $n-1$.

Пирамидальная сортировка

- ▶ Снова поменяем местами первый и $(n-1)$ -й элементы и вновь протолкнем первый элемент, но уже в куче из $n-2$ элементов и так далее. В результате получаем массив, отсортированный по неубыванию. Время проталкивания $O(\log_2 n)$, количество итераций $O(n)$, общая оценка – $O(n \cdot \log_2 n)$
- ▶ Метод не требует дополнительной памяти

Procedure Pr(r, q – числа):

«проталкивание» одного, где r – номер проталкиваемого элемента, q – верхнее значение индекса

$i = r, v = A[i]$ {Индекс рассматриваемого элемента и сам элемент}

$j = 2*i$ {Индекс элемента, с которым происходит сравнение}

$pp = \mathbf{False}$ {Считаем, что для элемента не найдено место в пирамиде}

While ($j \leq q$) **And Not** pp **Do**:

If $j < q$: **If** $A[j] > A[j+1]$: $j++$

{Сравниваем с меньшим элементом}

If $v \leq A[j]$: $pp = \mathbf{true}$

{Элемент находится на своем месте}

else: $A[i] = A[j], i = j, j = 2*i$

{Переставляем элемент и идем дальше по пирамиде}

$A[i] = v$

Procedure Sort:

$t = N \text{ Div } 2 + 1$ {Эта часть массива является пирамидой}

For $i = t - 1$ **DownTo** 1 **Do**: Pr(i, N)

{Строим пирамиду (только один раз)}

For $i = N$ **DownTo** 2 **Do**:

$w = A[1], A[1] = A[i], A[i] = w$

{Меняем местами 1-й и i -й элементы}

Pr(1, $i - 1$) {Проталкиваем 1-й элемент}

Ссылки

- ▶ <https://proglib.io/p/sort-gif/>
- ▶ <https://proglib.io/p/sort-algorithms/> (с vpn)

Бонус:

Ещё алгоритмы сортировки

Сортировка чет-нечет (англ. *odd-even sort*) – модификация метода сортировки обменом. Она основана на сравнении элементов стоящих на четных и нечетных позициях независимо друг от друга. Сложность – $O(n^2)$.

```
function oddEvenSort(a) :  
  for i = 0 to n - 1 :  
    if i mod 2 = 0 :  
      for j = 2 to n - 1 step 2 :  
        if a[j] < a[j - 1] :  
          swap(a[j - 1], a[j])  
    else :  
      for j = 1 to n - 1 step 2 :  
        if a[j] < a[j - 1] :  
          swap(a[j - 1], a[j])
```

Примечание. Преимущество этой сортировки – на нескольких процессорах она выполняется быстрее, так как четные и нечетные индексы сортируются параллельно. Н. Хаберман (N. Haberman) 1972 год.

Сортировка расческой (англ. *comb sort*) — модификация сортировки обменом. Основана на сравнении элементов на расстоянии. Сложность — $O(n^2)$, но стремится к $O(n \cdot \log_2 n)$. Является самым быстрым методом с квадратичной сложностью.

```
function combSort(a):  
    k = 1.3  
    jump = n  
    bool swapped = true  
    while jump > 1 and swapped:  
        if jump > 1:  
            jump /= k  
            swapped = false  
        for i = 0 to size - jump - 1:  
            if a[i + jump] < a[i]:  
                swap(a[i], a[i + jump])  
                swapped = true
```

Пояснения. Изначально расстояние между сравниваемыми элементами равно n/k , где $k=1.3$ – оптимальное число для этого алгоритма. Сортируем массив по этому расстоянию, затем уменьшаем его по тому же правилу. Когда расстояние между сравниваемыми элементами достигает единицы, сортировка элементов массива заканчивается обычным пузырьком.

Timsort – гибридный алгоритм сортировки, разработал Тим Петерсон в 2002.

Примечание. На массивах данных, которые содержат упорядоченный подмассивы, алгоритм Тима Петерса показывает себя намного лучше других сортировок. В настоящее время Timsort является стандартной сортировкой в Python и GNU Octave, реализован в OpenJDK 7 и Android JDK 1.5.

Идея алгоритма, он состоит из нескольких частей.

Шаг 1. Входной массив разделяется на подмассивы фиксированной длины, вычисляемой определённым образом.

Шаг 2. Каждый подмассив сортируется сортировкой вставками или сортировкой обменом (пузырьком), или другим методом.

Шаг 3. Отсортированные подмассивы объединяются в один массив с помощью модифицированной сортировки слияниями.

Автор алгоритма предлагает брать длину подмассива следующим образом: старшие 6 бит числа n и добавляется единица, если в оставшихся младших битах есть хотя бы один ненулевой.

<https://habrahabr.ru/post/251751/>

Доказательство некорректности Timsort

JSort – модификация пирамидальной сортировки Джейсона Моррисона. Алгоритм частично упорядочивает массив, строя на нем два раза кучу: один раз передвигая меньшие элементы влево, второй раз передвигая большие элементы вправо. Затем к массиву применяется сортировка вставками.

Пример. $A=(1, 2, 8, 15, 17, 20, 31, 32, 30, 2, 3, 5, 10, 11, 24)$. После первого построения пирамиды: $A=(1, 2, 5, 15, 2, 8, 11, 32, 30, 17, 3, 20, 10, 31, 24)$. После второго построения пирамиды: $A=(1, 2, 5, 15, 2, 3, 11, 20, 10, 17, 8, 24, 30, 31, 32)$. Далее сортировка вставками дает: $A=(1, 2, 2, 3, 5, 8, 10, 11, 15, 17, 20, 24, 30, 31, 32)$.

Терпеливая сортировка (англ. *patience sorting*) — алгоритм сортировки с худшей сложностью $O(n \cdot \log_2 n)$. Его краткое описание. Имеется массив $A[0..n-1]$, элементы которого нужно отсортировать по возрастанию. Разложим элементы массива по стопкам: для того чтобы положить элемент в стопку, требуется выполнение условия – новый элемент меньше элемента, лежащего на вершине стопки; либо создадим новую стопку справа и сделаем наш элемент её вершиной. Используется жадная стратегия. Каждый элемент кладётся в самую левую из возможных стопок, если же таковой нет, справа от существующих стопок создаётся новая. Для получения отсортированного массива после построения массива стопок, следует выполнить n шагов (здесь и далее нумерация шагов начинается с единицы). На i -м шаге выбирается стопка с наименьшим значением в вершине, извлекается из неё элемент и записывается в результирующий массив $Ans[0..n-1]$ на i -ю позицию.

Примечание. Создается список стеков для хранения стопок. При раскладывании элементов по стопкам для поиска самой левой подходящей стопки следует использовать бинарный поиск. Соответственно, поиск самой левой стопки имеет сложность $O(\log_2 p)$, где p — количество стопок (стеков). Таким образом, временная сложность раскладывания по стопкам не превышает $O(n \cdot \log_2 n)$. Для получения результирующего массива следует использовать бинарную кучу. На каждом шаге алгоритма необходимо извлекать из кучи стек с минимальной вершиной за $O(\log_2 p)$, где p — количество стеков в куче. Взять вершину выбранного стека и вернуть его в кучу за $O(\log_2 p)$. Получение отсортированного массива требует $O(n \cdot \log_2 n)$ времени.

Алгоритм терпеливой сортировки можно использовать для поиска наибольшей возрастающей подпоследовательности (НВП). Каждая стопка представляет собой убывающую последовательность, то есть длина НВП в пределах стопки равна единице, поэтому появление новой стопки можно понимать как увеличение длины НВП на единицу (изначально длина НВП равна единице). Поэтому длина НВП равна количеству стопок. Для получения НВП при формировании стопок проводятся следующие операции: каждый раз, положив элемент на вершину стопки, создается указатель на возможный предыдущий элемент (вершину ближайшей слева стопки). В конце для получения НВП требуется выполнить r шагов, начав с вершины самой правой стопки: на i -м шаге записать в результат на $(r-i)$ -ю позицию текущий элемент, перейти к предыдущему элементу по указателю, r – количество стопок.

Пример. Исходный массив 3, 1, 2, 4, 5, 0. Получаем стопки: 1-я – 0, 1, 3; 2-я – 2; 3-я – 4; 4-я – 5. Из Длина НВП равна 4. Из 5 (вершина 4-й стопки) указатель на вершину 3-й стопки – 4. Из 4 на 2. Из 2 на 1 в первой стопке, именно 1 была в вершине 1-й стопки на момент создания 2-й стопки. Последовательность: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$.